Design and Analysis of Algorithms

Assignment 1: Solution

Topic: Efficient search strategy for a student record system
Covers: Tasks 1 to 7 and Task 12. Tasks 8 to 11 (implementation, results, comparison, final recommendation) are in Python_Code.py, Output.png and Report.pdf.

Notation used throughout: n is the number of records, q is the number of searches, and "key" is the registration number being searched for. Registration numbers are unique. All logarithms are base 2.

Task 1: Algorithm Design

I chose three strategies built on three different ideas: brute force, decrease and conquer, and a space-time trade-off.

Approach A: Linear search (iterative, brute force)

Idea: Start at the first record and compare its registration number with the key. Move to the next record until the key is found or the records run out.
Preprocessing: None. It works on unsorted data.
Why I chose it: It is the natural baseline. When only one or two lookups are needed, nothing beats it, because it has no setup cost.

```
LinearSearch(A[0..n-1], key)
  for i <- 0 to n-1 do
    if A[i].regNo = key then return i
  return -1
```

Approach B: Binary search (recursive, decrease and conquer)

Idea: Keep the records sorted by registration number. Compare the key with the middle record. If they match, stop. Otherwise discard the half that cannot contain the key and repeat on the other half.
Preprocessing: Sort the records by registration number using merge sort, which costs about n log n comparisons.

Why I chose it: It shows how a one-time sorting cost buys much faster searches. The recursive form also gives a clean recurrence for Tasks 6 and 7. An iterative binary search would be the same algorithm, so I do not count it as a separate approach.

```
BinarySearch(A, low, high, key)
    if low > high then return -1
    mid <- floor((low + high) / 2)
    if A[mid].regNo = key then return mid
    else if key < A[mid].regNo then return BinarySearch(A, low, mid-1, key)
    else return BinarySearch(A, mid+1, high, key)
```

Approach C: Hash table lookup (transform and conquer, space-time trade-off)

Idea: Instead of narrowing a range, compute where the record should be. The hash function h(key) = key mod m sends the key straight to a bucket. Keys that land in the same bucket are kept in a chain. I take m as a prime near n / 0.75, so the load factor is $\alpha = n / m \approx 0.75$.
Preprocessing: Build the table by inserting all n records.
Why I chose it: It works on a different principle from the other two and trades extra memory for speed.

```
BuildHash(A[0..n-1], m)
    create table T[0..m-1] of empty lists
    for i <- 0 to n-1 do
        append (A[i].regNo, i) to T[A[i].regNo mod m]


HashSearch(T, m, key)
    for each entry e in T[key mod m] do
        if e.key = key then return e.index
    return -1
```

Task 2: Basic Operation

Linear search: The basic operation is the comparison A[i].regNo = key inside the loop. It runs once per pass, and the rest of the loop (the index increment) happens the same number of times, so counting comparisons captures the whole cost. In the worst case the count grows in step with n, so doubling the records doubles the work.

Binary search: The basic operation is the comparison of the key with A[mid], treated as one three-way comparison. Each call makes it once and then discards half the range, so the number of comparisons equals the number of calls. The count rises by only one each time n doubles.

Hash lookup: The basic operation is the key comparison while scanning a bucket's chain. Computing key mod m is a constant extra step that does not grow with n. The number of comparisons depends on how many keys share a bucket, which is set by the load factor α. If m grows with n, the count stays roughly constant. For the build phase, the basic operation is the insertion, which runs n times.

Comparisons suit all three because they are the most repeated step, and the cost of everything else is proportional to them.

Task 3: Time-Complexity Derivation

The input size is n, the number of records. For hashing, the table size m also matters.

Linear search. If the key is at position i, the search makes i comparisons. The best case is 1. The worst case is n, when the key is last or absent. If the key is present and equally likely to be at any position:

$C_{avg}(n) = (1/n) \cdot (1 + 2 + ... + n) = (1/n) \cdot n(n+1)/2 = (n+1)/2$

If the key is present with probability p, then $C_{avg} = p(n+1)/2 + n(1-p)$. There is no preprocessing, so the cost is $\Theta(n)$ per search.

Binary search. After the first comparison at most $\lfloor n/2 \rfloor$ records remain, and after k comparisons at most $\lfloor n/2^k \rfloor$. The range becomes empty when $n/2^k < 1$, that is, when $k > \log n$. So the worst case is $\lfloor \log n \rfloor + 1$ comparisons (proved formally in Task 7). For the average successful search, take $n = 2^k - 1$. Exactly $2^{(i-1)}$ keys are found at comparison i, so the total is Σ

$i \cdot 2^{(i-1)} = (k-1) \cdot 2^k + 1$. The average is therefore about $k - 1$, which is roughly $\log n - 1$. The preprocessing sort satisfies $M(n) = 2M(n/2) + n$, which gives about $n \log n$ comparisons.

Hash lookup. Assume the hash function spreads keys evenly over the m buckets. Then the expected chain length is $\alpha = n/m$.

- An unsuccessful search scans a whole chain, so it makes about α comparisons.
- A successful search makes 1 comparison plus one for each key placed earlier in the same bucket. The i-th inserted key has $(i-1)/m$ such keys on average, so the average is $1 + (1/n) \Sigma (i-1)/m = 1 + (n-1)/(2m) \approx 1 + \alpha/2$. With $\alpha = 0.75$ this is about 1.4 comparisons.
- Building the table takes n insertions at constant expected cost each, so $\Theta(n)$.

Preprocessing and searching together (q searches):

Approach	Preprocessing	Average per search	Total for q searches
Linear	none	$(n+1)/2$	$q(n+1)/2$, which is $\Theta(qn)$
Binary	about $n \log n$	about $\log n - 1$	$n \log n + q \log n$, which is $\Theta(n \log n + q \log n)$
Hash	about n	about $1 + \alpha/2$	$n + 1.4q$, which is $\Theta(n + q)$ expected

Break-even for n = 1,000,000. Binary search with sorting beats linear search once $2.0 \times 10^7 + 19q < 500{,}000q$, which gives $q > 40$ successful searches (about 20 if most searches fail). Hashing beats linear search after about 2 searches. So preprocessing is only worth paying for when searches are repeated.

Task 4: Best, Average and Worst Cases

Linear search. The best case is when the key is the very first record, which takes 1 comparison. The worst case is when the key is the last record or is not in the database, which takes n comparisons because every record must be ruled out. The average case, with the key equally likely to be anywhere, takes $(n+1)/2$ comparisons. The gap exists because the algorithm has no information about where the key lies, so the position alone decides how long it scans.

Binary search. The best case is when the key is exactly at the middle of the sorted array, found in 1 comparison. The worst case is when the key is absent or sits at the deepest level of the search tree, taking $\lfloor \log n \rfloor + 1$ comparisons. The average successful case is about $\log n - 1$, only one comparison fewer than the worst case. This happens because half of all records sit at the deepest level of the tree and a quarter at the level above, so most keys need close to the maximum number of steps.

Hash lookup. The best case is 1 comparison (or 0 for an absent key whose bucket is empty). The average case is about $1 + \alpha/2 \approx 1.4$ comparisons. The worst case is n comparisons, which happens when every key hashes to the same bucket and the chain degenerates into a linear scan. This can occur with a poor hash function, for example m = 1000 when registration numbers share a long common pattern. A prime table size reduces that risk.

Task 5: Asymptotic Analysis

Approach	Best	Average	Worst	Preprocessing
Linear	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	none
Binary	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(n \log n)$
Hash	$\Theta(1)$	$\Theta(1)$ expected	$\Theta(n)$	$\Theta(n)$

For every approach the running time is $\Omega(1)$ and $O(n)$ at worst, but what matters in practice is the typical case. Growth order from slowest to fastest is $O(1) < O(\log n) < O(n) < O(n \log n)$, and the sort is the only n log n step.

Approximate comparisons per search (and sorting cost):

n	Linear (average)	Binary (worst)	Hash (average)	Sort cost (n log n)
1,000	about 500	10	about 1.4	about 1.0×10^4
10,000	about 5,000	14	about 1.4	about 1.3×10^5
100,000	about 50,000	17	about 1.4	about 1.7×10^6
1,000,000	about 500,000	20	about 1.4	about 2.0×10^7

When the database grows 1,000 times, from 1,000 to 1,000,000 records, linear search becomes 1,000 times slower. Binary search only doubles from 10 to 20 comparisons. Hashing stays flat. For a university system that expects growth, this is the key point. Linear search stops being practical as the database grows. Binary search and hashing keep scaling, and their one-time setup cost is the price of that.

Task 6: Recurrence Relation (Recursive Binary Search)

Let n = high − low + 1 be the number of records in the current range.

- Original problem size: n.
- Subproblem size: After comparing with A[mid], the algorithm recurses on only one side. With mid = ⌊(low+high)/2⌋, the left side has ⌊(n−1)/2⌋ records and the right side has ⌊n/2⌋. The larger one, ⌊n/2⌋, gives the worst case.
- Number of recursive calls: 1. The algorithm never searches both halves.
- Work outside the recursive call: One comparison of the key with A[mid], plus constant arithmetic, so 1 basic operation.
- Base case: An empty range (n = 0) returns −1 with no comparison. A range of one record needs a single comparison, so T(1) = 1.

Putting these together, the worst-case recurrence is:

$T(n) = T(\lfloor n/2 \rfloor) + 1$ for $n > 1$, with $T(1) = 1$

The worst case follows the larger half each time and never finds the key until the range is exhausted.

For the sorting step in preprocessing, merge sort splits into two halves and merges them in linear time, so $M(n) = 2M(n/2) + n$ with $M(1) = 0$.

Task 7: Solving the Recurrence

Method: Iteration (backward substitution), checked by induction and by the Master Theorem. The method applies because each step shrinks the problem by a constant factor and adds constant work, so the recurrence unrolls cleanly when n = 2^k.

Iteration for n = 2^k:

T(2^k) = T(2^(k−1)) + 1
= T(2^(k−2)) + 2
= ...
= T(2^(k−i)) + i

Setting i = k gives T(2^k) = T(1) + k = 1 + k = log n + 1.

Induction for general n. Claim: T(n) = ⌊log n⌋ + 1. The base case n = 1 gives 1, which is correct. For n ≥ 2, assume the claim holds for smaller values:

T(n) = T(⌊n/2⌋) + 1 = (⌊log ⌊n/2⌋⌋ + 1) + 1 = (⌊log n⌋ − 1) + 2 = ⌊log n⌋ + 1

Master Theorem check. The recurrence has the form T(n) = aT(n/b) + f(n) with a = 1, b = 2 and f(n) = Θ(n^d) where d = 0. Since a = b^d (1 = 2^0), this is the case T(n) = Θ(n^d log n) = Θ(log n). For merge sort, a = 2, b = 2 and d = 1, so again a = b^d and M(n) = Θ(n log n).

Meaning. For 1,000,000 records the search needs at most 20 comparisons. Each doubling of the database adds only one more comparison. Even a database of one billion records would need only ⌊log 10^9⌋ + 1 = 30 comparisons.

Task 12: Day-4 Design Challenge

Starting point. In the original problem the number of searches was not fixed, so my recommended design was binary search on a sorted array. It guarantees at most 20 comparisons at one million records and needs no extra data structure, and the sort pays for itself after about 40 searches.

A. Impact Analysis

The load is now steady and large: 50,000 searches per hour is about 14 per second, or roughly 1.2 million searches a day (assuming the system runs around the clock), on a fixed 1,000,000 records. New records arrive only once a day.

Three assumptions are affected:

1. Preprocessing is no longer a big share of the cost. One daily rebuild is now shared by about 1.2 million searches.

2. Constant updates are no longer expected. Inserting into a sorted array costs O(n), which would hurt if records arrived all day. With one batch a day, the structure can simply be rebuilt.
3. Small per-search differences now add up. A saving of even a few operations per search is multiplied by 1.2 million.

Search frequency matters because total cost is preprocessing plus q times the per-search cost, and as q grows the second term dominates. The low update frequency matters because structures that are costly to build but fast to query become attractive: they are built once, then only read until the next day.

B. Evaluation of the Original Design

Binary search handles the new load easily. Each search takes about 19 comparisons on average, so a day's searches cost about 2.3×10^7 comparisons, which one machine absorbs without trouble at 14 requests per second. The daily sort costs about 2.0×10^7 operations, or about 17 per search once shared across the day's searches. That is acceptable. Re-sorting from scratch is not even necessary, because merging the day's new records into the sorted array costs only about 10^6.

Another approach can do better, though. A hash index needs about 1.4 comparisons per search and about 10^6 operations to build. That is roughly 16 times fewer operations than the binary-search route, at the price of extra memory proportional to n and no support for ordered lookups.

C. Adaptation: MODIFY

I keep the sorted array as the master copy and add a hash index from registration number to record position. The index is rebuilt once a day, after the new records have been merged in. Searches go through the index first. The sorted array stays available for ordered or range queries (for example, all students from one intake) and as a fallback with a guaranteed O(log n) bound.

What changes: one O(n) index build per day is added, and the per-search cost drops from Θ(log n) to Θ(1) expected. The worst case for hashing is O(n), but a prime table size and a load factor of 0.75 make that very unlikely. Daily cost moves from Θ(n log n + q log n) to Θ(n + q).

D. Evidence-Based Defence

Daily operation counts for 1,000,000 records and 1.2 million successful searches, counting one comparison or insertion as one operation:

Approach	Daily preprocessing	Daily searching	Total per day	Per search (amortized)
Linear	0	6.0×10^11	6.0×10^11	500,000
Binary (sort daily)	2.0×10^7	2.3×10^7	4.3×10^7	about 36
Hash index	1.0×10^6	1.7×10^6	2.7×10^6	about 2.2

Linear search is ruled out, since it needs about 14,000 times more operations than binary search. Between binary search and hashing the gap is about 16 times. In absolute terms both easily handle 14 requests per second, so the case for modifying is not that binary search fails. It is that hashing keeps search cost flat if the database or the request rate keeps growing, and the added cost is small: one linear-time rebuild per day and some extra memory. Keeping the sorted array preserves ordered queries and a safe fallback. The experimental results in the Report test these predictions on real runs.